

Teaching Notes: Searching & Sorting Algorithms (Java)

Duration: 2 Hours | Prerequisite: 1D and 2D Arrays (already taught)

SESSION PLAN (so you can manage time on the board)

Time	Topic
0:00 - 0:10	Recap of arrays + why we need searching/sorting
0:10 - 0:35	Linear Search (theory + code + dry run)
0:35 - 1:00	Binary Search (theory + code + dry run)
1:00 - 1:10	Break / Q&A
1:10 - 1:35	Bubble Sort (theory + code + dry run)
1:35 - 1:55	Selection Sort (theory + code + dry run)
1:55 - 2:00	Summary + comparison table

STEP 0: Recap (talk to class, no code yet)

Say this to students:

“You already know how to store data in an array. But storing data is useless if we can’t: 1. **Find** something inside it (Searching) 2. **Arrange** it in order (Sorting)

Example: A phone contact list is an array of names. To call ‘Rahul’, your phone *searches* the array. To show contacts A-to-Z, your phone *sorts* the array.”

Write on board:

Array + Searching + Sorting = Real Applications

PART A: SEARCHING

A1. Linear Search (Sequential Search)

Explain in one line: “Check every element one by one from start to end until you find what you want.”

Real life analogy: Finding your name in an unsorted attendance register — you check every line from top to bottom.

Steps to explain on board (write these exact steps): 1. Start from the first element (index 0) 2. Compare it with the target value 3. If it matches → found! Return the index 4. If not → move to the next index 5. Repeat until array ends 6. If nothing matches → return -1 (not found)

Code:

```
public class LinearSearchDemo {  
    public static void main(String[] args) {
```

```

int[] marks = {45, 67, 12, 89, 23, 90};
int target = 89;
int result = -1; // -1 means "not found" by default

// Step-by-step loop through every element
for (int i = 0; i < marks.length; i++) {
    System.out.println("Checking index " + i + " -> value = " + marks[i]);

    if (marks[i] == target) {
        result = i; // found! save the index
        break;      // stop searching further, no need to continue
    }

    if (result == -1) {
        System.out.println(target + " not found in the array.");
    } else {
        System.out.println(target + " found at index " + result);
    }
}

```

Explain each line while teaching: - `int[] marks = {...}` → our data (already known from 1D array class) - `int target = 89;` → the value we are searching for - `int result = -1;` → a variable to store the answer; -1 is a signal meaning “not found yet” - `for (int i = 0; i < marks.length; i++)` → visit every box in the array, one at a time - `if (marks[i] == target)` → compare current box’s value with what we want - `result = i; break;` → save position and immediately stop the loop (no point checking further) - final `if-else` → tell the user the result

Dry run on board (draw this table):

i	marks[i]	Match?
0	45	No
1	67	No
2	12	No
3	89	Yes → stop, return 3

Time Complexity to mention: Worst case checks all n elements → **O(n)**

A2. Binary Search

Explain in one line: “Works only on a SORTED array. Repeatedly cut the array in half to find the value fast.”

Real life analogy: Finding a word in a dictionary — you don’t check every page, you open the middle, decide left or right half, and repeat.

Important rule to emphasize loudly: “Binary Search ONLY works if the array is already sorted!”

Steps to explain on board: 1. Take `low = 0` and `high = array.length - 1` 2. Find `mid = (low + high) / 2` 3. If `array[mid] == target` → found! 4. If `target < array[mid]` → target is in the LEFT half → move `high = mid - 1` 5. If `target > array[mid]` → target is in the RIGHT half → move `low = mid + 1` 6. Repeat until `low > high` (means not found)

Code:

```

public class BinarySearchDemo {

```

```

    public static void main(String[] args) {
        int[] numbers = {10, 20, 30, 40, 50, 60, 70}; // must be
sorted!

        int target = 50;

        int low = 0;
        int high = numbers.length - 1;
        int result = -1;

        while (low <= high) {
            int mid = (low + high) / 2; // middle index

            System.out.println("low=" + low + " high=" + high + "
mid=" + mid + " value=" + numbers[mid]);

            if (numbers[mid] == target) {
                result = mid; // found!
                break;
            } else if (target < numbers[mid]) {
                high = mid - 1; // search left half
            } else {
                low = mid + 1; // search right half
            }
        }

        if (result == -1) {
            System.out.println(target + " not found.");
        } else {
            System.out.println(target + " found at index " +
result);
        }
    }
}

```

Explain each line while teaching: - low and high → the current boundaries we are searching within - mid = (low + high) / 2 → middle point of current boundary - comparing numbers[mid] with target decides which half to keep searching - high = mid - 1 → throw away right half (including mid) - low = mid + 1 → throw away left half (including mid) - loop stops when low > high, meaning boundaries crossed = not found

Dry run on board:

Array: {10, 20, 30, 40, 50, 60, 70}, target = 50

low	high	mid	numbers[mid]	Decision
0	6	3	40	50 > 40 → go right, low=4
4	6	5	60	50 < 60 → go left, high=4
4	4	4	50	Match! Found at index 4

Time Complexity to mention: Cuts array in half every time → **O(log n)** — much faster than linear search for large data.

PART B: SORTING

B1. Bubble Sort

Explain in one line: “Compare two neighboring elements again and again, swap if they are in wrong order. Bigger values ‘bubble up’ to the end.”

Real life analogy: Arranging students by height — you compare two people standing next to each other, swap if the shorter one is behind, and keep repeating down the line.

Steps to explain on board: 1. Compare element at index j and j+1 2. If array[j] > array[j+1] → swap them 3. Move to next pair, repeat till end of array (this is one “pass”) 4. After each full pass, the largest remaining value settles at the end 5. Repeat the whole process for n-1 passes

Code:

```
public class BubbleSortDemo {
    public static void main(String[] args) {
        int[] arr = {64, 34, 25, 12, 22};

        System.out.println("Before sorting:");
        printArray(arr);

        // Outer loop -> number of passes
        for (int i = 0; i < arr.length - 1; i++) {

            // Inner loop -> comparing neighbors in each pass
            for (int j = 0; j < arr.length - 1 - i; j++) {

                if (arr[j] > arr[j + 1]) {
                    // swap arr[j] and arr[j+1]
                    int temp = arr[j];
                    arr[j] = arr[j + 1];
                    arr[j + 1] = temp;
                }
            }
            System.out.println("After pass " + (i + 1) + ":");
            printArray(arr);
        }

        System.out.println("After sorting:");
        printArray(arr);
    }

    // helper method to print array (reuse – students already know
    // methods+arrays)
    static void printArray(int[] arr) {
        for (int val : arr) {
            System.out.print(val + " ");
        }
        System.out.println();
    }
}
```

Explain each line while teaching: - Outer loop i → controls how many full passes we make - Inner loop j → walks through the array comparing pairs - arr.length - 1 - i → shrinks each pass because the end part is already sorted from previous passes - int temp = arr[j]; arr[j]=arr[j+1]; arr[j+1]=temp; → the classic 3-step swap (very important — write this on board separately and explain why we need temp) - printArray() → a reusable helper method (connects back to functions/methods concept)

Dry run on board:

Array: {64, 34, 25, 12, 22}

Pass 1: compare(64,34)→swap, compare(64,25)→swap, compare(64,12)→swap, compare(64,22)→swap Result: {34, 25, 12, 22, 64} (64 bubbled to the end)

Pass 2: {25, 12, 22, 34, 64} Pass 3: {12, 22, 25, 34, 64} Pass 4: {12, 22, 25, 34, 64} (already sorted, no more swaps)

Time Complexity to mention: Two nested loops → **O(n²)**

B2. Selection Sort

Explain in one line: “Find the SMALLEST element in the unsorted part and put it at the front. Repeat.”

Real life analogy: Sorting playing cards in your hand — you look through all cards, pick the smallest, place it first, then repeat with the remaining cards.

Steps to explain on board: 1. Assume the first element is the smallest (minIndex = i) 2. Compare it with every other element after it 3. If a smaller value is found, update minIndex 4. After checking all, swap array[i] with array[minIndex] 5. Move to next index, repeat

Code:

```
public class SelectionSortDemo {
    public static void main(String[] args) {
        int[] arr = {29, 10, 14, 37, 13};

        System.out.println("Before sorting:");
        printArray(arr);

        for (int i = 0; i < arr.length - 1; i++) {

            int minIndex = i; // assume current position holds the
smallest

            // search for the actual smallest in the remaining
unsorted part

            for (int j = i + 1; j < arr.length; j++) {
                if (arr[j] < arr[minIndex]) {
                    minIndex = j; // found a new smaller value,
update index
                }
            }

            // swap only if a smaller element was found elsewhere
            if (minIndex != i) {
                int temp = arr[i];
                arr[i] = arr[minIndex];
                arr[minIndex] = temp;
            }

            System.out.println("After step " + (i + 1) + ":");
            printArray(arr);
        }

        System.out.println("After sorting:");
        printArray(arr);
    }

    static void printArray(int[] arr) {
        for (int val : arr) {
            System.out.print(val + " ");
        }
        System.out.println();
    }
}
```

Explain each line while teaching: - minIndex = i → we temporarily “assume” the current position is smallest - inner loop j → scans the rest of the array to check for a smaller value - minIndex = j → updates our “best guess” whenever a smaller value is found - swap happens **only once per outer loop pass** (this is the key difference from Bubble Sort, which swaps many times per pass)

Dry run on board:

Array: {29, 10, 14, 37, 13}

Step 1: smallest in whole array = 10 (index 1) → swap with index 0 → {10, 29, 14, 37, 13} Step 2: smallest in remaining {29,14,37,13} = 13 (index 4) → swap with index 1 → {10, 13, 14, 37, 29} Step 3: smallest in remaining {14,37,29} = 14, already in place → {10, 13, 14, 37, 29} Step 4: smallest in remaining {37,29} = 29 → swap → {10, 13, 14, 29, 37}

Time Complexity to mention: Two nested loops → **$O(n^2)$** , but fewer swaps than Bubble Sort.

FINAL SUMMARY TABLE (write on board at the end)

Algorithm	Works on	Best Use Case	Time Complexity
Linear Search	Sorted or unsorted array	Small data / unsorted data	$O(n)$
Binary Search	Sorted array only	Large sorted data, fast lookup	$O(\log n)$
Bubble Sort	Any array	Teaching purpose, simple logic	$O(n^2)$
Selection Sort	Any array	Fewer swaps needed	$O(n^2)$

Closing line for class: > “Searching helps us FIND data. Sorting helps us ARRANGE data. Together, they are the foundation of almost every real-world application — from your contacts app to Google Search.”

HOMEWORK / PRACTICE QUESTIONS (optional, give at end)

1. Modify Linear Search to count how many comparisons it took.
2. Modify Binary Search to work on a descending sorted array.
3. Sort an array of **student names** (Strings) using Bubble Sort.
4. Trace Selection Sort by hand for the array {5, 1, 4, 2, 8} and write each step.